

Convert JSON SQL queries from an Oracle database to a PostgreSQL database

by Pinesh Singal and Lokesh Gurram | on 29 DEC 2023 | in [Advanced \(300\)](#), [Customer Solutions](#), [PostgreSQL compatible](#), [RDS for Oracle](#), [RDS for PostgreSQL](#), [Technical How-to](#) | [Permalink](#) | [Comments](#) | [Share](#)

Customers use semi-structured or unstructured data storage for various business use cases, which is schema-less and flexible in nature. One such type of semi-structured data is [JavaScript Object Notation \(JSON\)](#). JSON stores data in the form of KEY, VALUE, LIST, and ARRAY formats.

Oracle Database stores JSON object data in [CLOB data type](#) columns. PostgreSQL provides the JSON (text) and JSONB (binary) columns for storing [JSON](#) data, and you can also choose to store JSON data in a TEXT column if it's not RFC 8259 compliant.

In this post, we present a solution that helps you convert JSON SQL queries from an Oracle database to a PostgreSQL database. Our solution primarily focuses on converting JSON-related data into TABLE (ROW and COLUMN) format and vice versa using Oracle and PostgreSQL. You can use this solution when migrating from on-premises Oracle or [Amazon Relational Database Service \(Amazon RDS\) for Oracle](#) to [Amazon RDS for PostgreSQL](#) or [Amazon Aurora PostgreSQL-Compatible Edition](#).

Solution overview

This solution works for source Oracle Database version 12.2 and above and target RDS for PostgreSQL or Aurora PostgreSQL database instances with version 9.5 and above.

As part of the JSON query conversion, we use the following functions in Oracle:

- [JSON_TABLE](#) – Transforms the JSON data into TABLE format
- [JSON_ARRAYAGG](#) – Aggregates the JSON format
- [JSON_OBJECT](#) – Transposes the TABLE data into JSON format

We use the following functions in PostgreSQL:

- [JSONB_BUILD_OBJECT](#) – Creates and returns a JSON object from a variadic parameter list consisting of alternating keys and values
- [JSONB_AGG](#) – An aggregate function that returns a JSON array containing all the values in a group
- [JSONB_ARRAY_ELEMENTS](#) – Expands the top-level JSON array into a set of JSON values

Note that the SQL standard [JSON_ARRAYAGG](#) and [JSON_OBJECT](#) functions are available and supported from PostgreSQL 16 onwards.

In the following sections, we show the step-by-step conversion of JSON-based SQL queries from Oracle to PostgreSQL.

We use the following sample JSON data in our examples:

```
{
  "metadata" : {
    "upperLastNameFirstName" : "ABC XYZ",
    "upperEmailAddress" : "abc@example.com",
    "profileType" : "P"
  },
  "data" : {
    "onlineContactId" : "032323323",
    "displayName" : "Abc, Xyz",
    "firstName" : "Xyz",
    "lastName" : "Abc",
    "emailAddress" : "abc@example.com",
    "productRegistrationStatus" : "Not registered",
    "positionId" : "0100",
    "arrayPattern" : " -",
    "account" : {
      "companyId" : "SMGE",
      "businessUnitId" : 7,
      "accountNumber" : 42000,

```

Prerequisites

To follow along with this post, you need the following prerequisites:

- An [active AWS account](#)
- An on-premises Oracle DB instance (source database) up and running with version 12.2 and above
- An RDS for PostgreSQL or Aurora PostgreSQL DB instance (target database) up and running with version 9.5 and above

We use the [AWS Schema Conversion Tool](#) (AWS SCT) to [convert the source Oracle database functions to PostgreSQL](#). The conversion produces PostgreSQL-supported migrated [SQL files](#). You can run these .SQL files on your PostgreSQL DB instance.

Convert JSON data into TABLE format

In this section, we discuss two options to convert JSON data into TABLE format:

- Convert JSON data available in tables
- Convert static JSON data passed to the JSON table function

These options explain how to convert Oracle and PostgreSQL JSON data into TABLE format for data that is stored in tables and static values passed from the application layer.

Convert Oracle JSON data available in tables

The SQL/JSON function JSON_TABLE creates a relational view of JSON data. It maps the result of a JSON data evaluation into relational rows and columns. You can query the result returned by the function as a virtual relational table using SQL. The main purpose of JSON_TABLE is to create a row of relational data for each object inside a JSON array and output JSON values from within that object as individual SQL column values.

You can specify JSON_TABLE only in the FROM clause of a SELECT statement. The function first applies a JSON path expression, called a *row path expression*, to the supplied JSON data. The JSON value that matches the row path expression is called a *row source*, which generates a row of relational data. The COLUMNS clause evaluates the row source, finds specific JSON values within the row source, and returns those JSON values as SQL values in individual columns of a row of relational data.

JSON_TABLE has the following parameters:

- **Expression** – Passing the JSON document or column that has JSON content
- **JSON path expression** – The path or hierarchy of the JSON document
- **ERROR clause** – Error handling when the respective JSON field doesn't exist
- **COLUMN list** – A list of columns that maps the JSON fields in the JSON object

The following table shows how to read the TABLE data using JSON_TABLE with our sample data.

Key	Hierarchy or path to be used to get the value	Value
profileType	metadata -> profileType	"P"
positionId	data -> positionId	"0100"
accountNumber	data -> account -> accountNumber	42000

From this table, we can understand the following:

- To get the `profileType` value, the hierarchy or path to use is `metadata -> profile type`
- To get the `positionId` value, the hierarchy or path to use is `data -> positionId`
- To get the `accountNumber` value, the hierarchy or path to use is `data -> account -> accountNumber`

Let's insert our sample JSON data into the table `aws_test_table`, with column `json_doc` of CLOB data type. Use the provided SQL query to convert the selected JSON fields into TABLE format.

1. Connect to your source Oracle database using SQL Developer or another preferred tool.
2. Run the following command to create a test table:

```
CREATE TABLE aws_test_table (id NUMBER, created_on DATE DEFAULT SYSDATE, modified_on DATE, json_doc CLOB);
```

3. Run the following command to insert sample JSON data into the table:

```
REM INSERTING INTO EXPORT_TABLE
SET DEFINE OFF;
INSERT INTO aws_test_table (ID,CREATED_ON,MODIFIED_ON,json_doc)
VALUES (1,to_date('02-AUG-2022 12:30:14','DD-MON-YYYY HH24:MI:SS'),to_date('02-AUG-2022 12:30:14','DD-MON-YYYY HH24:MI:SS'),
'{"metadata" : {
  "upperLastNameFirstName" : "ABC XYZ",
  "upperEmailAddress" : "abc@example.com",
  "profileType" : "P"
},
"data" : {
  "onlineContactId" : "032323323",
  "displayName" : "Abc, Xyz",
  "firstName" : "Xyz",
  "lastName" : "Abc",
  "emailAddress" : "abc@example.com",

```

```
"productRegistrationStatus" : "Not registered",
"positionId" : "0100",
"..." : "..."
```

4. Run the following SQL query to convert data into ROWS and COLUMNS:

```
SELECT
  parent_account_number,
  account_number,
  business_unit_id,
  position_id
FROM
  aws_test_table aws,
  JSON_TABLE (
    json_doc,
    '$' ERROR ON ERROR COLUMNS (
      parent_account_number NUMBER PATH '$.data.account.parentAccountNumber',
      account_number NUMBER PATH '$.data.account.accountNumber',
      business_unit_id NUMBER PATH '$.data.account.businessUnitId',
      position_id VARCHAR2 (4) PATH '$.data.positionId'
    )
  ) AS sc;
```

From the preceding query, we can understand that the JSON_TABLE function has the following parameters:

- **JSON_DOC** – This is the `json_doc` column, which contains the JSON content.
- **\$** – The path is defined as `$`. By default, `$` indicates all fields in the JSON document. For example, `$.account[*]` means all fields under the `account` key (refer to the sample data). `$.metadata[*]` means all fields under the `metadata` key.
- **ERROR ON ERROR** – This is the error handling mechanism. It has three variants:
 - **NULL ON ERROR** – Returns null when an error occurs. This is the default.
 - **ERROR ON ERROR** – Returns the appropriate Oracle error when an error occurs.
 - **DEFAULT literal ON ERROR** – Returns a literal when an error occurs. If the data type of the value returned by this function is VARCHAR2, then you must specify a text literal. If the data type is NUMBER, then you must specify a numeric literal.
- **COLUMNS** – This is the list of column names along with data type and path. If the respective path exists in JSON, it returns the value associated to that field. If not, an error is handled.

The following screenshot shows the result of the preceding query.

123 PARENT_ACCOUNT_NUMBER	123 ACCOUNT_NUMBER	123 BUSINESS_UNIT_ID	ABC POSITION_ID
32000	42000	7	0100

Convert PostgreSQL JSON data available in a table

Oracle has the JSON_TABLE function for transposing JSON data into TABLE format. However, PostgreSQL does not currently have the SQL/JSON JSON_TABLE function, so instead we must read the data directly from the JSON document to convert it to a table. To illustrate this, we use the following sample JSON data.

1. Connect to the target PostgreSQL database using [pgAdmin](#) or another preferred tool.
2. Run the following command to create a test table:

```
CREATE TABLE aws_test_pg_table(id INT, created_on DATE, modified_on DATE, json_doc JSONB);
```

3. Run the following command to insert the JSON data into the test table:

```
INSERT INTO aws_test_pg_table(id,created_on,modified_on,json_doc)
VALUES(1,now(),now(),'{
  "metadata" : {
    "upperLastNameFirstName" : "ABC XYZ",
    "upperEmailAddress" : "abc@example.com",
    "profileType" : "P"
  },
  "data" : {
    "onlineContactId" : "032323323",
    "displayName" : "Abc, Xyz",
    "firstName" : "Xyz",
    "lastName" : "Abc",
```

```
"emailAddress" : "abc@example.com",
"productRegistrationStatus" : "Not registered",
"positionId" : "0100",
"arrayPattern" : " -",
"account" : {
  "companyId" : "SMGE",
```

4. Run the following SQL query to convert the data into ROWS and COLUMNS:

```
SELECT *
FROM (
SELECT
(json_doc::jsonb->'data'->'account'->'parentAccountNumber') AS parentAccountNumber,
(json_doc::jsonb->'data'->'account'->'accountNumber') AS accountNumber,
(json_doc::jsonb->'data'->'account'->'businessUnitId') AS businessUnitId,
(json_doc::jsonb->'data'->'positionId') AS positionId
FROM aws_test_pg_table) d ;
```

Using a single greater than (`->`) operator returns the value defined for that JSON data type. For example, key `positionId` and value `"0100"` . However, if you want the key value as an INT or TEXT, you can't directly typecast JSON data type. You can use the double greater than (`->>`) operator to return the key value in INT or TEXT format.

In the following SQL query, typecasting is allowed:

```
SELECT *
FROM (
SELECT (json_doc::jsonb->'data'->'account'->>'parentAccountNumber')::INTEGER AS parentAccountNumber,
(json_doc::jsonb->'data'->'account'->>'accountNumber')::INTEGER AS accountNumber,
(json_doc::jsonb->'data'->'account'->>'businessUnitId')::INTEGER AS businessUnitId,
(json_doc::jsonb->'data'->>'positionId')::TEXT AS positionId
FROM aws_test_pg_table
) d ;
```

Using double greater than (`->>`) returns the actual value defined for that key. For example, key `positionId` and value `0100` (without double quotes). Typecasting is allowed when we use `->>` .

In the preceding case, `parentAccountNumber` , `accountNumber` , and `businessUnitId` are typecasted to INT, and `positionId` is typecasted to TEXT.

The following tables compare the role of `>` and `>>` .

Following data is with single greater (>). Each column is in JSON data type and can't be converted to another data type.

parentaccountnumber jsonb	accountnumber jsonb	businessunitid jsonb	positionid jsonb
32000	42000	7	"0100"
32001	42001	6	"0090"

Following data is with double greater (>>). Each column allows type CAST based on the column value, for example, INTEGER

parentaccountnumber integer	accountnumber integer	businessunitid integer	positionid character varying
32000	42000	7	0100
32001	42001	6	0090

Convert static Oracle JSON data passed to the JSON table function

We can use the same JSON_TABLE function to convert static JSON data into TABLE format. See the following example code:

```
SELECT * FROM
JSON_TABLE (
  '{
    "accounts": [{
      "accountNumber": 42000,
      "parentAccountNumber": 32000,
      "businessUnitId": 7
    }, {
      "accountNumber": 42001,
      "parentAccountNumber": 32001,
      "businessUnitId": 6
    }]
  }',
  '$.accounts[*]' ERROR ON ERROR COLUMNS (
    parent_account_number PATH '$.parentAccountNumber',
    account_number PATH '$.accountNumber',
```

```
business_unit_id PATH '$.businessUnitId'
)
\.
```

The following screenshot shows our output.

ABC PARENT_ACCOUNT_NUMBER	ABC ACCOUNT_NUMBER	ABC BUSINESS_UNIT_ID
32000	42000	7
32001	42001	6

Convert static PostgreSQL JSON data passed to the FROM clause query

The JSONB_ARRAY_ELEMENTS function helps convert static JSON data (passed to the FROM clause query) into TABLE format. The method we used in the previous section doesn’t work for converting the static JSON data into TABLE format.

In the following SQL query, typecasting is not allowed (fails with a syntax error):

```
SELECT
(jsonb_array_elements(b.jc -> 'accounts') -> 'accountNumber') accountNumber,
(jsonb_array_elements(b.jc -> 'accounts') -> 'businessUnitId') businessUnitId,
(jsonb_array_elements(b.jc -> 'accounts') -> 'parentAccountNumber') parentAccountNumber
FROM (
SELECT '{
  "accounts": [{
    "accountNumber": 42000,
    "parentAccountNumber": 32000,
    "businessUnitId": "7"
  }, {
    "accountNumber": 42001,
    "parentAccountNumber": 32001,
    "businessUnitId": "6"
  }]
}'::jsonb AS jc) b;
```

Using -> returns the value defined for that key. For example, key businessUnitId and value "7" . Typecasting is not allowed to primitive data types when we use -> .

In the following SQL query, typecasting is allowed:

```
SELECT
(jsonb_array_elements(b.jc ->> 'accountNumber')::INTEGER accountNumber,
(jsonb_array_elements(b.jc ->> 'businessUnitId')::INTEGER businessUnitId,
(jsonb_array_elements(b.jc ->> 'parentAccountNumber')::INTEGER parentAccountNumber
FROM (
SELECT '{
  "accounts": [{
    "accountNumber": 42000,
    "parentAccountNumber": "32000",
    "businessUnitId": "7"
  }, {
    "accountNumber": 42001,
    "parentAccountNumber": 32001,
    "businessUnitId": "6"
  }]
}'::jsonb AS jc) b;
```

Using ->> returns the actual value defined for that key. For example, key businessUnitId and value 7 (without double quotes). Typecasting is allowed when we use ->> .

In the preceding case, parentAccountNumber , accountNumber , and businessUnitId are typecast to INT.

The following tables compare the roles of -> and ->> .

The following data is with single greater (>). Each column is in JSON data type and can't be converted into another data type.

accountnumber	businessunitid	parentaccountnumber
42000	"7"	32000
42001	"6"	32001

The following data is with double greater (>>). Each column allows type CAST based on the column value, for example, INTEGER.

accountnumber	businessunitid	parentaccountnumber
42000	7	32000
42001	6	32001

Convert TABLE format data into JSON format

In this section, we demonstrate how to convert Oracle and PostgreSQL TABLE format data into JSON format data.

Oracle TABLE format

We can use the JSON_OBJECT function to convert the TABLE data into JSON format (a key-value pair). This has the following parameters:

- **KEY and a key expression** – For example, `account`
- **VALUE and a value expression** – For example, `column name FROM table`, which gets the value from the table

The SQL and JSON function JSON_ARRAYAGG is an aggregate function. It takes as its input a column of SQL expressions, converts each expression to a JSON value, and returns a single JSON array that contains those JSON values. This is primarily used to convert multiple rows into a single array in JSON format. The following code is an example SQL query:

```
SELECT json_arrayagg(json_object('SerialNo' value col))
FROM (
  SELECT 1 col FROM dual
  UNION ALL
  SELECT 2 FROM dual
  UNION ALL
  SELECT 3 FROM dual
  UNION ALL
  SELECT 4 FROM dual
);
```

We get the following output.

```
[
  {
    "SerialNo": 1
  },
  {
    "SerialNo": 2
  },
  {
    "SerialNo": 3
  },
  {
    "SerialNo": 4
  }
]
```

In this example, the data in the table is converted into a key-value pair and becomes aggregated as a list using JSON_ARRAYAGG.

The following output was generated from the query in the previous section:

PARENT_ACCOUNT_NUMBER	ACCOUNT_NUMBER	BUSINESS_UNIT_ID
32000	42000	7
32001	42001	6

We can use the following SQL query to convert this output:

```
SELECT
  json_object(
    'data' value json_arrayagg(
      json_object('positionId' value position_id)
    ),
    'account' value json_arrayagg(
      json_object(
```

```

        'parentAccountNumber' value parent_account_number,
        'accountNumber' value account_number,
        'businessUnitId' value business_unit_id
    )
)
)
FROM
(
    SELECT
        parent_account_number,
        account_number,

```

We get the following output.

```

{
  "data": [
    {
      "positionId": "0100"
    }
  ],
  "account": [
    {
      "parentAccountNumber": 32000,
      "accountNumber": 42000,
      "businessUnitId": 7
    }
  ]
}

```

PostgreSQL TABLE format data

Since PostgreSQL 16, PostgreSQL supports the JSON_OBJECT and JSON_ARRAYAGG functions, which are part of the SQL/JSON standard. For earlier versions of PostgreSQL, you'll need to use some other functions to support these queries.

The following PostgreSQL functions read the TABLE data and convert it into JSON format:

- **JSONB_BUILD_OBJECT** – This works the same as Oracle JSON_OBJECT. However, the syntax is different:
`JSONB_BUILD_OBJECT (key1, value1, key2, value2, key3, value3, keyn, valuen) .`
- **JSONB_AGG** – This works the same as Oracle JSON_ARRAYAGG. The syntax also is the same as Oracle.

In the following SQL query, we convert static JSON data from TABLE format into JSON:

```

SELECT
jsonb_build_object ('accountCounts',
    jsonb_agg(
        jsonb_build_object('accountNumber',accountNumber
        , 'businessUnitId',businessUnitId
        , 'parentAccountNumber',parentAccountNumber
    ))) FROM
(SELECT
(jsonb_array_elements(b.jc -> 'accounts') -> 'accountNumber')::INTEGER accountNumber,
(jsonb_array_elements(b.jc -> 'accounts') -> 'businessUnitId')::INTEGER businessUnitId,
(jsonb_array_elements(b.jc -> 'accounts') -> 'parentAccountNumber')::INTEGER parentAccountNumber
FROM (
SELECT '{
    "accounts": [{
        "accountNumber": 42000,
        "parentAccountNumber": 32000,
        "businessUnitId": 7
    }, {
        "accountNumber": 42001,

```

We get the following output.

```
{
  "accountCounts": [
    {
      "accountNumber": 42000,
      "businessUnitId": 7,
      "parentAccountNumber": 32000
    },
    {
      "accountNumber": 42001,
      "businessUnitId": 6,
      "parentAccountNumber": 32001
    }
  ]
}
```

In the following SQL query, we convert JSON data stored in tables in TABLE format into JSONB:

```
SELECT
jsonb_build_object ('accountCounts',
  jsonb_agg(
    jsonb_build_object('accountNumber', a.accountNumber
      , 'businessUnitId', a.businessUnitId
      , 'parentAccountNumber', a.parentAccountNumber
    ))) FROM
(
  SELECT (json_doc::jsonb->'data'->'account'->>'parentAccountNumber')::INTEGER AS parentAccountNumber,
  (json_doc::jsonb->'data'->'account'->>'accountNumber')::INTEGER AS accountNumber,
  (json_doc::jsonb->'data'->'account'->>'businessUnitId')::INTEGER AS businessUnitId,
  (json_doc::jsonb->'data'->>'positionId')::VARCHAR AS positionId
FROM aws_test_pg_table) a;
```

We get the following output.

```
{
  "accountCounts": [
    {
      "accountNumber": 42000,
      "businessUnitId": 7,
      "parentAccountNumber": 32000
    },
    {
      "accountNumber": 42001,
      "businessUnitId": 6,
      "parentAccountNumber": 32001
    }
  ]
}
```

Comparing JSON SQL query conversion between Oracle and PostgreSQL

In the following screenshots, we compare the conversion options we've discussed between Oracle and PostgreSQL.

The following queries show the option using static JSON data to convert into TABLE format.

Oracle SQL Query

```
SELECT * FROM JSON_TABLE ('{
  "accounts": [{
    "accountNumber": 42000,
    "parentAccountNumber": 32000,
    "businessUnitId": 7
  }, {
    "accountNumber": 42001,
```

Postgres SQL Query

```
SELECT
(jsonb_array_elements(b.jc -> 'accounts') ->>
  'parentAccountNumber')::INTEGER parentAccountNumber,
(jsonb_array_elements(b.jc -> 'accounts') ->>
  'accountNumber')::INTEGER accountNumber,
(jsonb_array_elements(b.jc -> 'accounts') ->>
  'businessUnitId')::INTEGER businessUnitId
```

```
FROM (
SELECT '{
"accounts": [ {
  "accountNumber":42000,
  "parentAccountNumber":32000,
  "businessUnitId":7
}, {
  "accountNumber":42001,
  "parentAccountNumber":32001,
  "businessUnitId":6
}] }':jsonb AS jc ) b;
```

PARENT_ACCOUNT_NUMBER	ACCOUNT_NUMBER	BUSINESS_UNIT_ID
32000	42000	7
32001	42001	6

Oracle SQL Query

Postgres SQL Query

```
SELECT
(json_doc::jsonb -> 'data' -> 'account'
->> 'parentAccountNumber')::INTEGER AS parentAccountN
(json_doc::jsonb -> 'data' -> 'account'
->> 'accountNumber')::INTEGER AS accountNumber,
(json_doc::jsonb -> 'data' -> 'account'
->> 'businessUnitId')::INTEGER AS businessUnitId,
(json_doc::jsonb -> 'data'
->> 'positionId')::VARCHAR AS positionId
FROM aws_test_pg_table;
```

Results...

Enter a SQL expression to filter results (u

PARENT_ACCOUNT_NUMBER	ACCOUNT_NUMBER	BUSINESS_UNIT_ID	POSITI
32000	42000	7	0100
32001	42001	6	0090

Oracle SQL Query

Postgres SQL Query:

```
SELECT jsonb_build_object(
    'accountCounts', jsonb_agg(jsonb_build_object(
        'accountNumber', a.accountNumber,
        'businessUnitId', a.businessUnitId,
        'parentAccountNumber', a.parentAccountNumber
    )))
FROM (
SELECT
    ( json_doc::jsonb -> 'data' -> 'account'
      -> 'parentAccountNumber'):: INTEGER AS parent
    ( json_doc::jsonb -> 'data' -> 'account'
      -> 'accountNumber')::INTEGER AS accountNumber
    ( json_doc::jsonb -> 'data' -> 'account'
      -> 'businessUnitId')::INTEGER AS businessUnit
    ( json_doc::jsonb -> 'data'
      -> 'positionId')::VARCHAR AS positionId
FROM aws_test_pg_table) a;
```

Oracle Output

```
{
  "data": [
    {
      "positionId": "0100"
    }
  ],
  "account": [
    {
      "parentAccountNumber": 32000,
      "accountNumber": 42000,
      "businessUnitId": 7
    }
  ]
}
```

PostgreSQL Output

```
{
  "data": [
    {
      "positionId": "0100"
    }
  ],
  "accountCounts": [
    {
      "parentAccountNumber": 32000,
      "accountNumber": 42000,
      "businessUnitId": 7
    }
  ]
}
```

Clean up

Because this solution uses manual code conversion, cleanup of code, policies, or roles is not required. Delete the target database if it's not needed anymore.

Summary

In this post, we explained the various functions to create and process JSON data in Oracle and PostgreSQL databases. We also showed how to convert Oracle JSON-related code (JSON data into ROW and COLUMN or TABLE format and vice versa) into PostgreSQL manually.

PostgreSQL has ongoing work upstream that can help simplify moving JSON data from Oracle to PostgreSQL. Specifically, PostgreSQL is working to include SQL/JSON standard functions, which includes `JSON_TABLE`, in a future release.

If you have any questions or comments about this post, please share your thoughts in the comments section.

About the Authors



Pinesh Singal is a Sr. Lead Consultant with AWS ProServe, GCC India. He works as a Database Migration Consultant, assisting and enabling customers to migrate servers and databases to AWS. He has architected and designed many successful databases and migration solutions addressing challenging business requirements. His primary focus is homogeneous and heterogeneous migrations of on-premises databases to Amazon RDS along with complete migration automation solutions.



Lokesh Gurram is a Lead Consultant with AWS ProServe, GCC India. He assists and enables customers to migrate servers and databases to AWS. He loves working backward and automating manual processes to enhance the user experience with least Business As Usual. His primary focus is homogeneous and heterogeneous migrations of on-premises databases to Amazon RDS along with complete migration automation solutions.

TAGS: [JSON](#), [Oracle](#), [Oracle to PostgreSQL](#), [PostgreSQL](#), [SCT](#)

Comments

0 Comments

Login ▾

G

Start the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name



Share

[Best](#) [Newest](#) [Oldest](#)

Be the first to comment.

[Subscribe](#)

[Privacy](#)

[Do Not Sell My Data](#)